Lecture 1: Introduction to Data Science

LSE ME314: Introduction to Data Science and Machine Learning (https://github.com/me314-lse)

2026-07-13

Daniel de Kadt

Slides last updated: 13 July 2026


오징어 게임 2
456억, 동심의 게임은 끝나지 않았다
ONLY ON NETFLIX | 12월 26일 공개

What Questions Can We Ask?

Imagine you're a member of Netflix's c-suite. What kind of questions might you have about Squid Game?

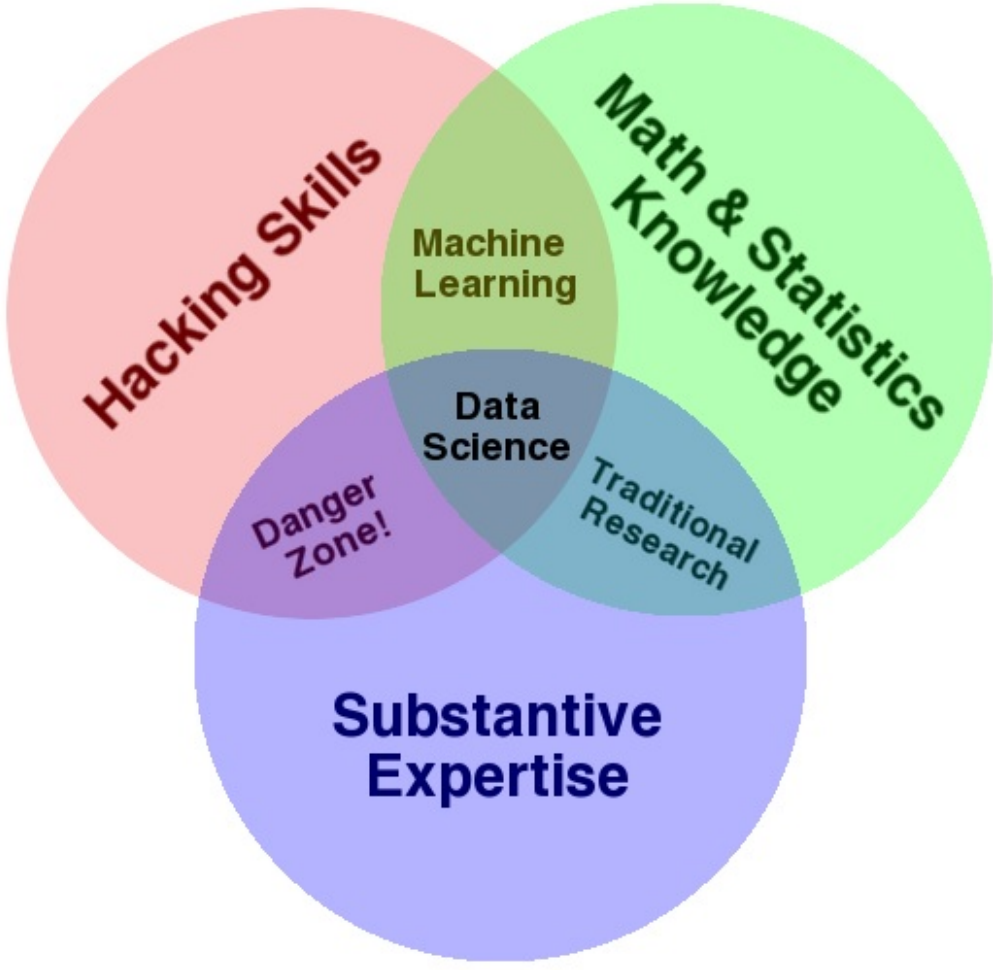
What Questions Can We Ask?

Imagine you're a member of Netflix's c-suite. What kind of questions might you have about Squid Game?

- ➜ Did viewers like Squid Game?
- ➜ Which viewers most liked Squid Game?
- ➜ What types of viewers engaged with Squid Game?
- ➜ Did Squid Game increase Netflix viewership?
- ➜ What was the $ value of Squid Game?
- ➜ Who should we advertise or push Squid Game to?
- ➜ How many $ should we spend on advertising Squid Game?
- ➜ Should we invest in Season 4?
- ➜ What can we learn from Squid Game about the types of shows that are successful?

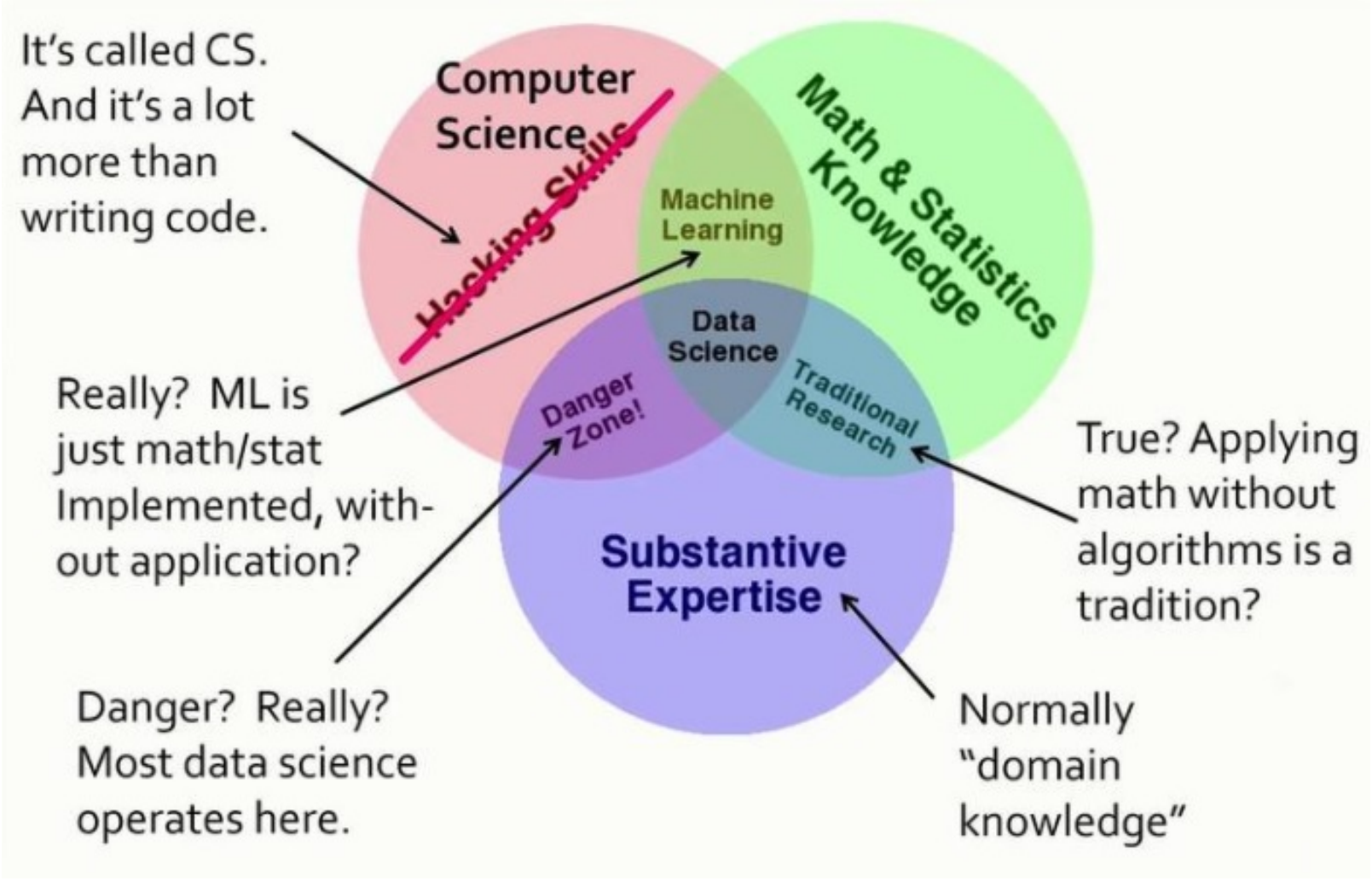
What is Data Science?

What is Data Science?



Source: Drew Conway, Data Science Venn Diagram

What is Data Science?



Source: Jeffrey Ullman, What Is Data Science? A Turing Award Winner Shares His View

What is Data Science?

What's missing from this venn diagram?

What is Data Science?

What's missing from this venn diagram?

1. **Data**:
   - ➜ Typically at scale
   - ➜ Is not value- or decision-free
   - ➜ Must be domain-specific

What is Data Science?

What's missing from this venn diagram?

1. **Data**:
   - ➜ Typically at scale
   - ➜ Is not value- or decision-free
   - ➜ Must be domain-specific

2. **Science**:
   - ➜ Principles, practices, and ethics
   - ➜ Norms (that may or may not be good)
   - ➜ Always domain-specific

What is Data Science?

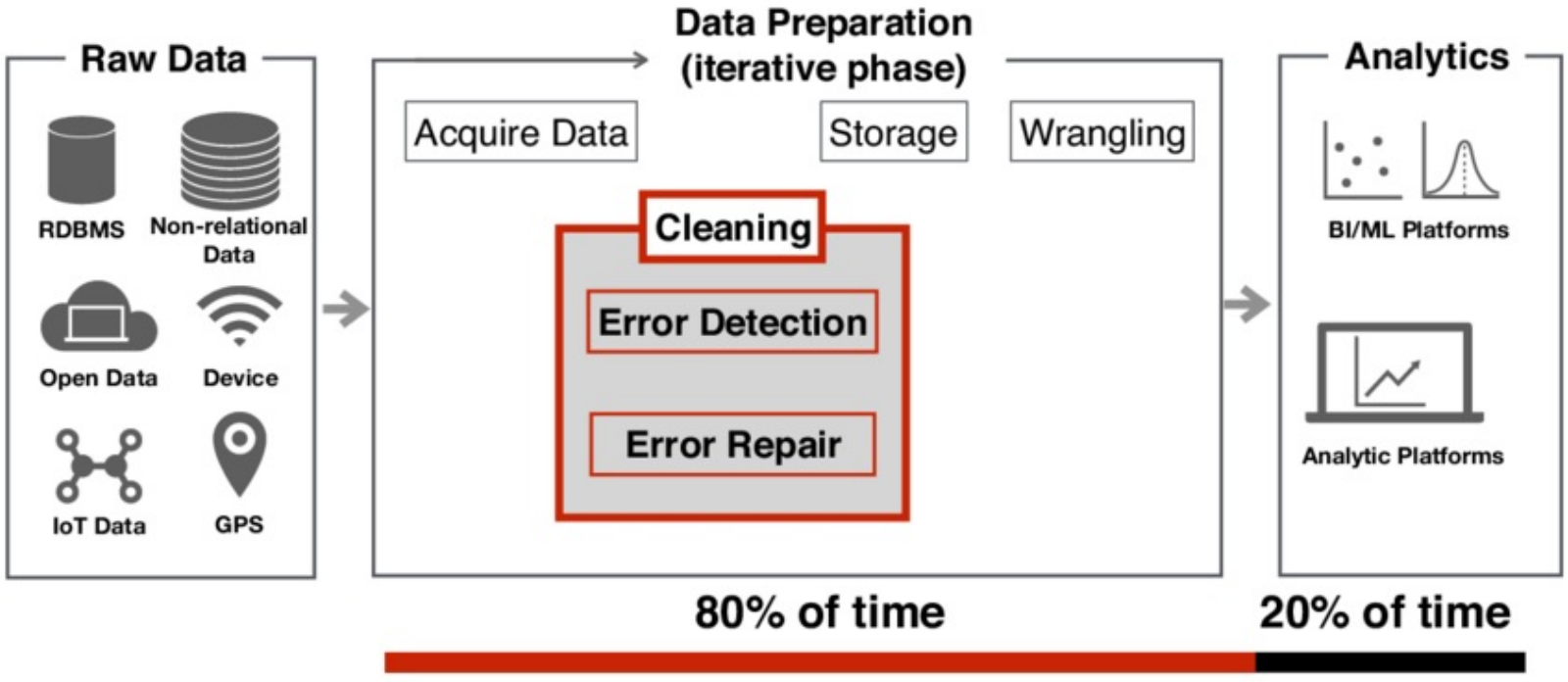
What's missing from this venn diagram?

1. **Data**:
   - ➜ Typically at scale
   - ➜ Is not value- or decision-free
   - ➜ Must be domain-specific

2. **Science**:
   - ➜ Principles, practices, and ethics
   - ➜ Norms (that may or may not be good)
   - ➜ Always domain-specific

Let's try that again: Data science is the marriage of **scientific skills** to **data** (at scale), all of which are *always* domain-specific.

Data Science Landscape



Note: We will spend most of our time on the 20%, but some of lectures 1-3 will cover the 80%.

Source: Larysa Visengeriyeva, Data Science Workflow

Data Science Landscape

1. Engineers
   - ➜ Backend data engineers
   - ➜ Analytics engineers
   - ➜ Machine learning engineers

Data Science Landscape

1. Engineers
   - ➜ Backend data engineers
   - ➜ Analytics engineers
   - ➜ Machine learning engineers

2. Scientists
   - ➜ Data scientists
   - ➜ Machine learning scientists
   - ➜ Research scientists
   - ➜ Behavioral scientists
   - ➜ Economists
   - ➜ UX researchers

Data Science Landscape

1. Engineers
   - ➜ Backend data engineers
   - ➜ Analytics engineers
   - ➜ Machine learning engineers

2. Scientists
   - ➜ Data scientists
   - ➜ Machine learning scientists
   - ➜ Research scientists
   - ➜ Behavioral scientists
   - ➜ Economists
   - ➜ UX researchers

3. Analysts
   - ➜ Business intelligence analysts
   - ➜ Data analysts

Data Science Landscape – Examples

As an Apple, you have a unique opportunity to truly create the destiny and pathway that fuels you each day and drives you to deliver the best results for yourself as well as your clients. As a Data Analyst, you will act as a strategic partner to your clients and media team, solving complex problems through metrics and analysis. Our Data Analysts have a curious, logical mindset with an interest in advertising and digital marketing analytics, working internally across disciplines to deliver the most impactful integrated media solution for our clients.

Key Responsibilities:

- Collaborate with internal teams to translate business needs into measurement plans and their implementations via website/ad-server tracking setups and data flows
- Democratize data and insights through data integration and the use of visualization tools
- Work with media teams to create marketing plans and ensure measurement requirements are built into the media structure
- Turn business requirements into metrics and implement tracking solutions to measure those metrics
- Learn what makes an effective media plan so you can be the voice of the data for the media teams and clients to guide media planning and optimization
- Build dashboards and supporting SQL database setups for weekly, monthly, and quarterly reports and analyses for clients
- Provide ad-hoc data science-driven regression analyses that discover the connections not discernible from a dashboard or spreadsheet
- Create clean, effective, and actionable presentations for clients and internal teams
- Excellent analytical skills and the ability to plainly communicate results to clients and internal teams
- Lead and contribute to processes (technical or business) improvements for reporting and analysis
- Diligently see solutions through to the end and be meticulous on QA

Desired Skills/Qualifications:

- Bachelor's Degree with 3.0 or higher GPA or equivalent work experience
- 0-2 years of marketing or analytics experience

Example recruitment listing for “Data Analyst” position.

Source: LinkedIn, 12 July 2026.

Data Science Landscape – Examples

About The Role

As a Data Engineer, you will be an early member of the Data Science & Analytics team building the foundation to scale analytics across our organization. You will collaborate with key stakeholders in Engineering, Product, GTM and other areas to build scalable solutions to transform data into key metrics reporting and insights. You will be responsible for ensuring teams have access to reliable, accurate metrics that can scale with our company's growth. You will also lead your own projects to enable self-serve insights to help teams make data-driven decisions.

Responsibilities

- Understand the data needs of stakeholder teams in terms of key data models and reporting, and translate that into technical requirements
- Define, build and manage key data pipelines in dbt that transform raw logs into canonical datasets
- Establish high data integrity standards and SLAs to ensure timely, accurate delivery of data
- Develop insightful and reliable dashboards to track performance of core metrics that will deliver insights to the whole company
- Build foundational data products, dashboards and tools to enable self-serve analytics to scale across the company
- Influence the future roadmap of Product and GTM teams from a data systems perspective
- Become an expert in our organization's data models and the company's data architecture

You Might Be a Good Fit If You Have

- 5+ years of experience as a Data Engineer or similar Data Science & Analytics roles, preferably partnering with GTM and Product leads to build and report on key company-wide metrics.

Example recruitment listing for "Data Engineer" position.

Source: LinkedIn, 12 July 2026.

Data Science Landscape – Examples

Build and maintain predictive models for customer preferences and individualized treatment effect models that inform business strategy and investment decisions.

Collaborate with Engineering to build scalable data pipelines, feature stores, and real-time serving infrastructure that support ongoing model development and experimentation.

Partner with engineering teams to deploy data science models and solutions into production across email, site, and paid media channels, ensuring models translate from development into customer-facing impact.

Translate complex analytical and modeling results into clear, actionable recommendations for leadership and cross-functional stakeholders, influencing strategy through evidence rather than intuition.

Basic Qualifications

- 5+ years of data querying languages (e.g. SQL), scripting languages (e.g. Python) or statistical/mathematical software (e.g. R, SAS, Matlab, etc.) experience
- 4+ years of data scientist experience
- Experience with statistical models e.g. multinomial logistic regression
- Bachelor's degree

Preferred Qualifications

- 2+ years of data visualization using AWS QuickSight, Tableau, R Shiny, etc. experience

Example recruitment listing for "Data Scientist (Machine Learning)" position.

Source: LinkedIn, 12 July 2026.

Data Science Landscape – Examples

You Will:

- Apply causal inference methods where experimentation isn't feasible
- Develop models and analyses that inform pricing, segmentation, and revenue optimization
- Design, run, and analyze A/B experiments
- Partner with product, engineering, and finance to translate ambiguous business questions into measurement frameworks
- Build and maintain dashboards, reporting, and analytical tooling that support ongoing decision-making

You Have:

- 3+ years of experience as a data scientist, applied scientist, economist, or related field; OR a PhD in Economics, Statistics, Computer Science, or related quantitative field
- Proficiency in SQL
- Proficiency with Python or R
- Strong foundation in experimentation and causal inference, including A/B test design and quasi-experimental methods
- Strong communication skills across technical and non-technical stakeholders
- Comfort building dashboards and recurring reporting

Example recruitment listing for "Data Scientist (Causal Inference)" position.

Source: LinkedIn, 12 July 2026.

Data Science Landscape – Examples

You will help define how we structure trade-offs like customer experience vs. operational efficiency, speed vs. cost, prediction accuracy vs. business impact, short-term metric movement vs. long-term marketplace health, and automation vs. human judgment. You'll prototype, experiment, influence architecture, and ensure we operationalize models and insights that actually move business metrics — not just analyses that look good offline.

The Impact You Will Make

- Serve as a technical thought leader in Data Science — defining principles, frameworks, and best practices for how Wonder uses data, experimentation, and machine learning to improve customer, marketplace, and business outcomes.
- Mentor and coach a growing team of Data Scientists and contribute to career development and technical excellence across the group.
- Lead the exploration of interconnected marketplace systems, recognizing feedback loops between customer behavior, fulfillment reliability, ETA accuracy, pricing, supply planning, product experience, and business performance.
- Develop causal inference and experimentation frameworks that help Wonder understand which product, operational, and marketplace changes truly drive business impact.
- Partner with engineering to drive architecture decisions for shared data layers, feature pipelines, modeling APIs, experimentation infrastructure, and production ML services.
- Define and implement robust experimentation strategies for changes that move business metrics in high-noise environments.
- Champion business-impact-driven data science, integrating causal inference, experimentation, risk-aware modeling, and scalable production ML systems that learn and adapt.

Example recruitment listing for "Data Scientist (Full Stack)" position.

Source: LinkedIn, 12 July 2026.

Data Science Problems

Recall, this course will focus on the **scientists** part of the landscape (the 20%). What kind of problems do those people try to solve?

Data Science Problems

Recall, this course will focus on the **scientists** part of the landscape (the 20%). What kind of problems do those people try to solve?

1. Descriptive problems:
   - ➜ Did viewers like Squid Game?
   - ➜ Can we group our Squid Game viewers into different types?

Data Science Problems

Recall, this course will focus on the **scientists** part of the landscape (the 20%). What kind of problems do those people try to solve?

1. Descriptive problems:
   - ➜ Did viewers like Squid Game?
   - ➜ Can we group our Squid Game viewers into different types?

2. Predictive problems:
   - ➜ Which viewers are likely to enjoy Squid Game?
   - ➜ What other shows are Squid Game viewers likely to enjoy?

Data Science Problems

Recall, this course will focus on the **scientists** part of the landscape (the 20%). What kind of problems do those people try to solve?

1. Descriptive problems:
   - ➜ Did viewers like Squid Game?
   - ➜ Can we group our Squid Game viewers into different types?

2. Predictive problems:
   - ➜ Which viewers are likely to enjoy Squid Game?
   - ➜ What other shows are Squid Game viewers likely to enjoy?

3. Causal problems:
   - ➜ Did Squid Game cause viewers to stay or churn?
   - ➜ Was Squid Game worth the investment?

Data Science Problems

Recall, this course will focus on the **scientists** part of the landscape (the 20%). What kind of problems do those people try to solve?

1. Descriptive problems:
   - ➜ Did viewers like Squid Game?
   - ➜ Can we group our Squid Game viewers into different types?

2. Predictive problems:
   - ➜ Which viewers are likely to enjoy Squid Game?
   - ➜ What other shows are Squid Game viewers likely to enjoy?

3. Causal problems:
   - ➜ Did Squid Game cause viewers to stay or churn?
   - ➜ Was Squid Game worth the investment?

Good data scientists *scope* their problem, implement the right kinds of *solutions*, and deliver *meaningful learning*.

Data Science Solutions: Measurement and Regression

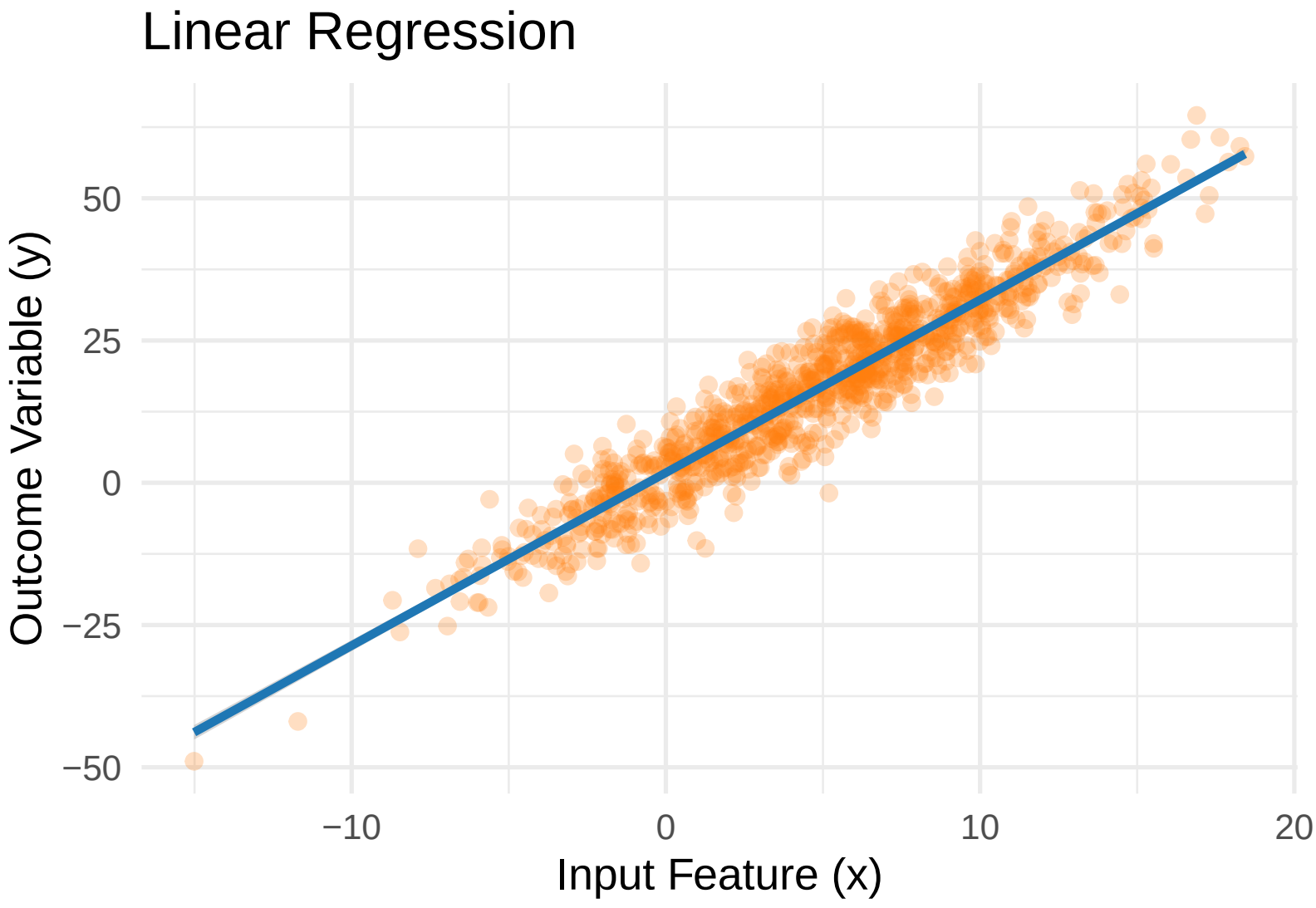
Measurement:

- ➜ You define a concept and develop a valid and reliable measure
- ➜ Examples $\longrightarrow$ surveys, psychometrics, and measurement models

Regression:

- ➜ The foundational tool for statistical comparison
- ➜ You connect an output variable Y to input features via a chosen functional ('parametric') form
- ➜ Examples $\longrightarrow$ ordinary least squares ('linear regression'), generalized linear models, and generalized additive models

Data Science Solutions: Measurement and Regression



Data Science Solutions: Machine Learning Models

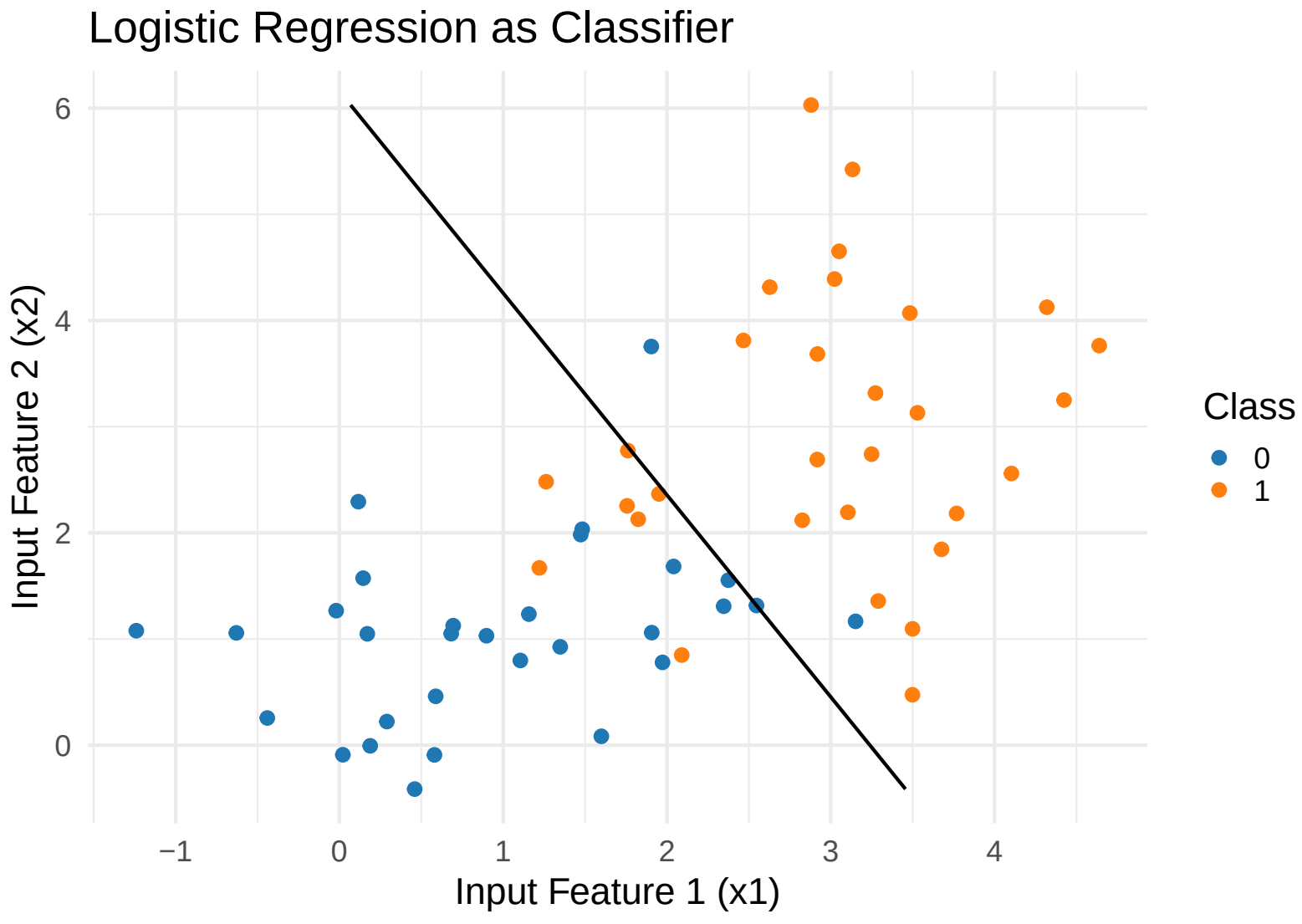
Supervised learning:

- ➜ You fit a model on some ('training') data, then apply the model to unseen ('out of sample') data
- ➜ You provide a set of output labels, and the model assigns those output labels to new inputs
- ➜ Examples $\longrightarrow$ linear regression ('LPM'), logistic regression, decision trees, random forests, and neural networks

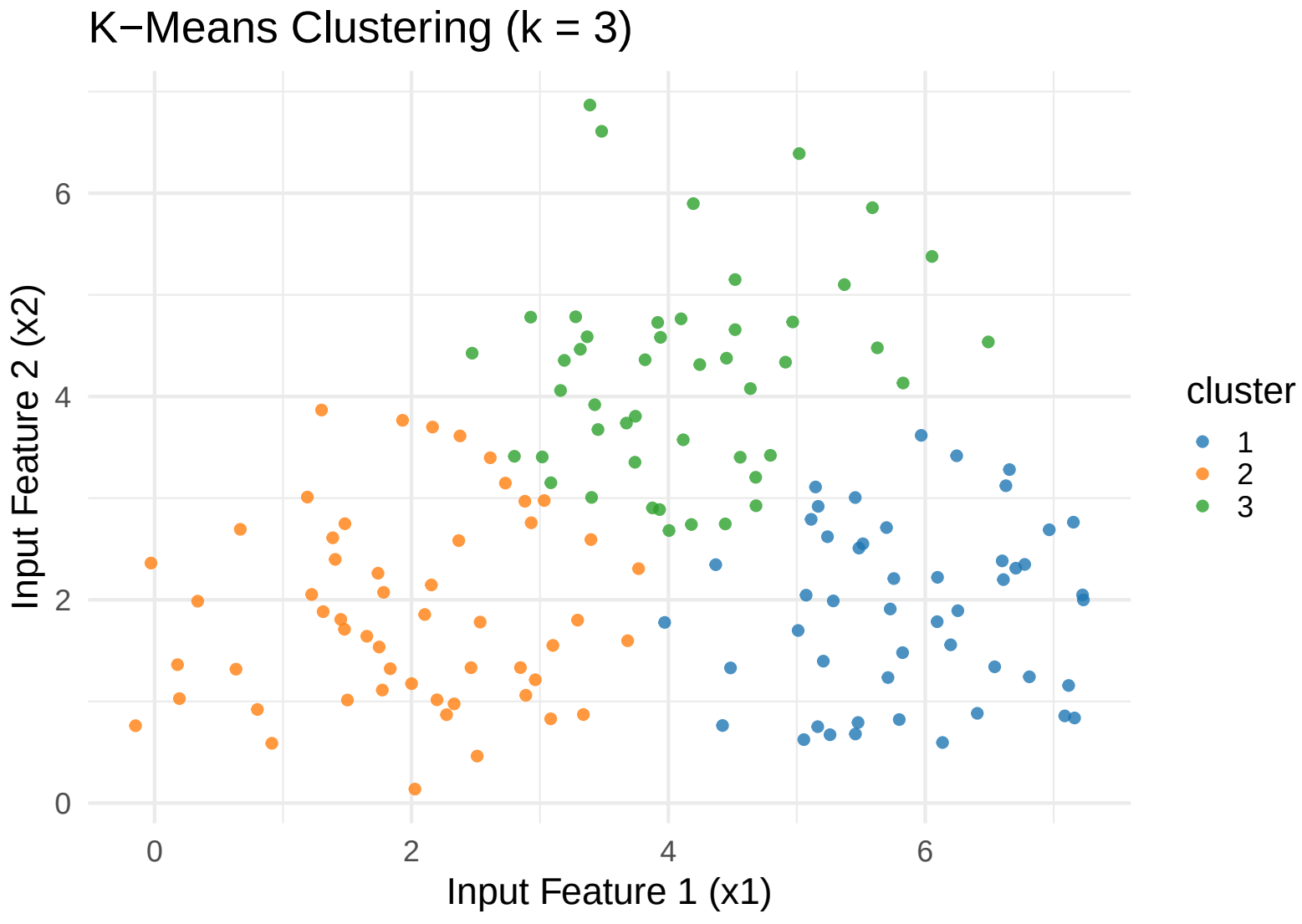
Unsupervised learning:

- ➜ You provide a set of inputs, and the model tries to find patterns in those inputs
- ➜ No training required (but my use cross-validation or other out-of-sample techniques)
- ➜ Example $\longrightarrow$ clustering, dimension reduction, topic modeling, word embeddings

Data Science Solutions: Machine Learning Models



Data Science Solutions: Machine Learning Models



Data Science Solutions: Causal Designs

Experiments ("A/B tests"):

- ➜ You control assignment of some causal feature (A or B)
- ➜ Examples $\longrightarrow$ classical, clustered, and cross-overs

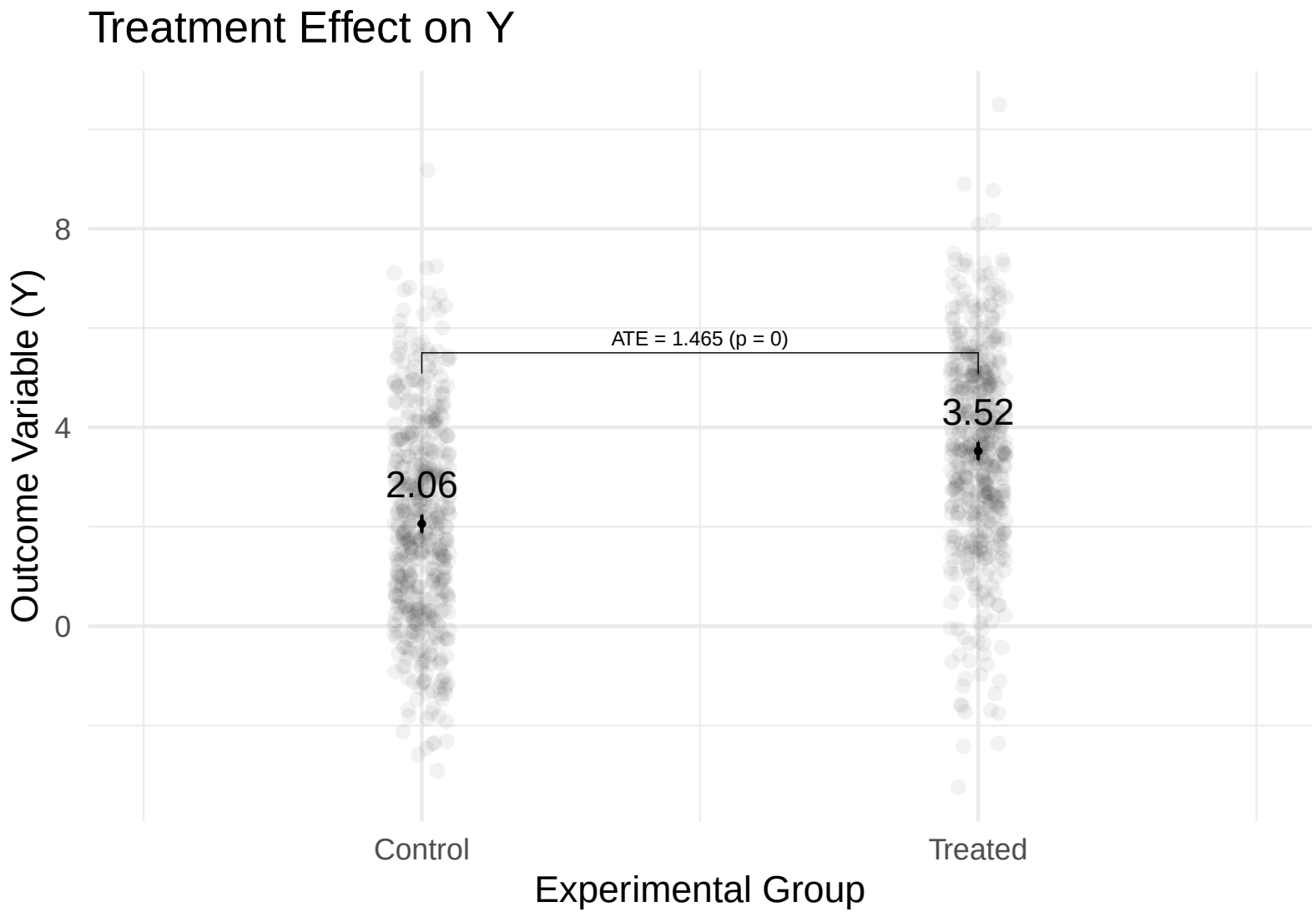
Observational designs:

- ➜ You assume something about the assignment mechanism
- ➜ Examples $\longrightarrow$ selection on observables, instrumental variables, and regression discontinuity (kinda)

Model-based analyses:

- ➜ You assume something about your outcome variable
- ➜ Examples $\longrightarrow$ regression discontinuity (kinda), difference-in-differences, synthetic control

Data Science Solutions: Causal Designs



Data Science Learnings

What kind of learnings might we look for from any given problem and solution?

Data Science Learnings

What kind of learnings might we look for from any given problem and solution?

1. Retrospection:
   - ➜ Example: "This is what happened"
   - ➜ Useful for understanding past events and planning for the future.

Data Science Learnings

What kind of learnings might we look for from any given problem and solution?

1. Retrospection:
   - ➜ Example: "This is what happened"
   - ➜ Useful for understanding past events and planning for the future.

2. Predictions:
   - ➜ Example: "This is what we expect to happen"
   - ➜ Useful for targeting and forecasting.

Data Science Learnings

What kind of learnings might we look for from any given problem and solution?

1. Retrospection:
   - ➜ Example: "This is what happened"
   - ➜ Useful for understanding past events and planning for the future.

2. Predictions:
   - ➜ Example: "This is what we expect to happen"
   - ➜ Useful for targeting and forecasting.

3. Explanations:
   - ➜ Example: "This is why/how something happened"
   - ➜ Useful for understanding what's going on.

Data Science Learnings

What kind of learnings might we look for from any given problem and solution?

1. Retrospection:
   - ➜ Example: "This is what happened"
   - ➜ Useful for understanding past events and planning for the future.

2. Predictions:
   - ➜ Example: "This is what we expect to happen"
   - ➜ Useful for targeting and forecasting.

3. Explanations:
   - ➜ Example: "This is why/how something happened"
   - ➜ Useful for understanding what's going on.

Good data scientists think carefully about the *type of learnings* they hope to extract from any given project.

Computational Landscape

The Command Line

We will spend a lot of this class 'talking to' our computers.

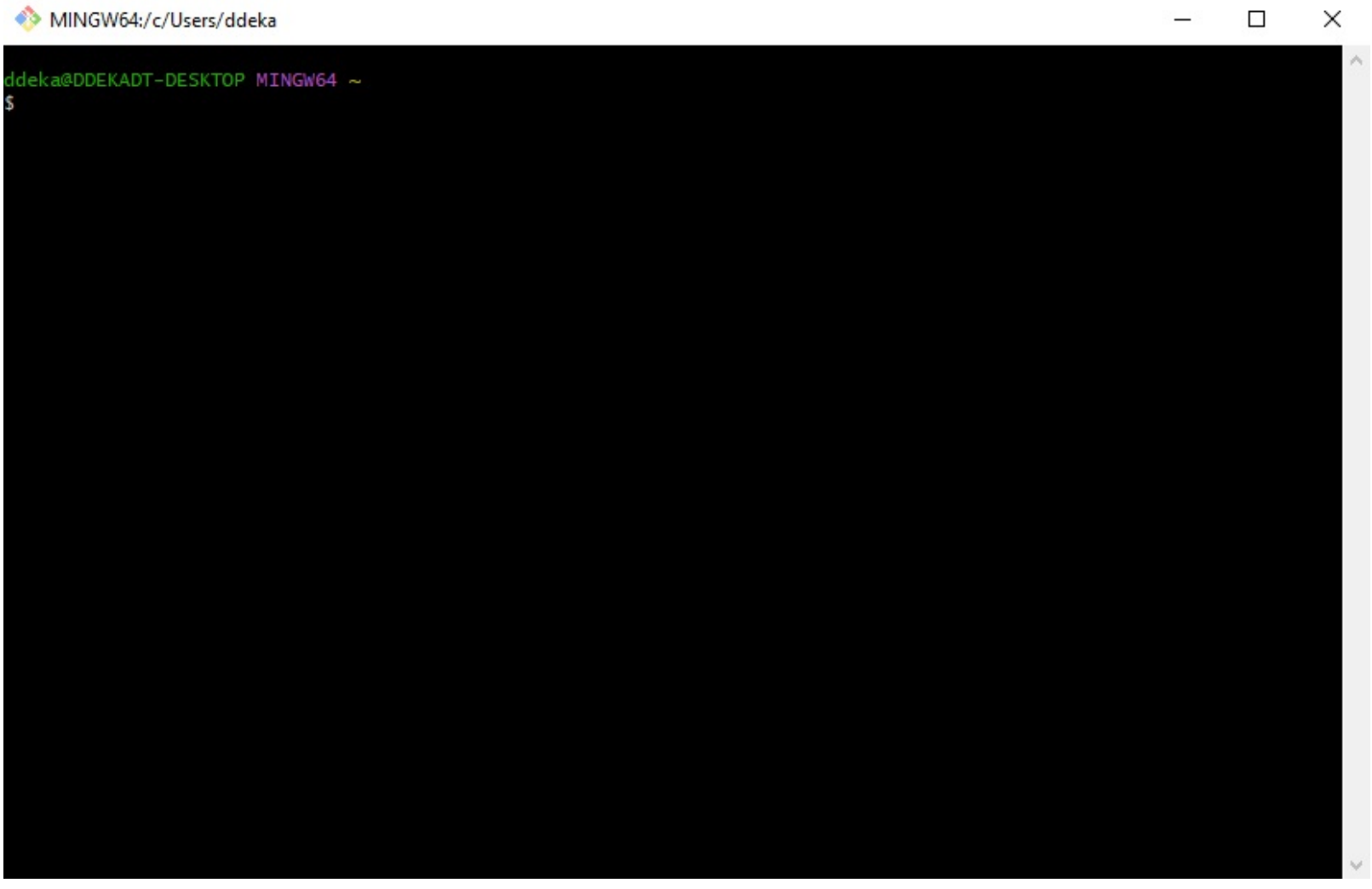
One way to do that directly is via the 'command line':

In Mac or Linux, this is called the 'Terminal'.

In Windows, use Git Bash or PowerShell (not command prompt).

We can use the command line to navigate around our folders, execute Git commands, execute scripts, and interact with our files.

The Command Line

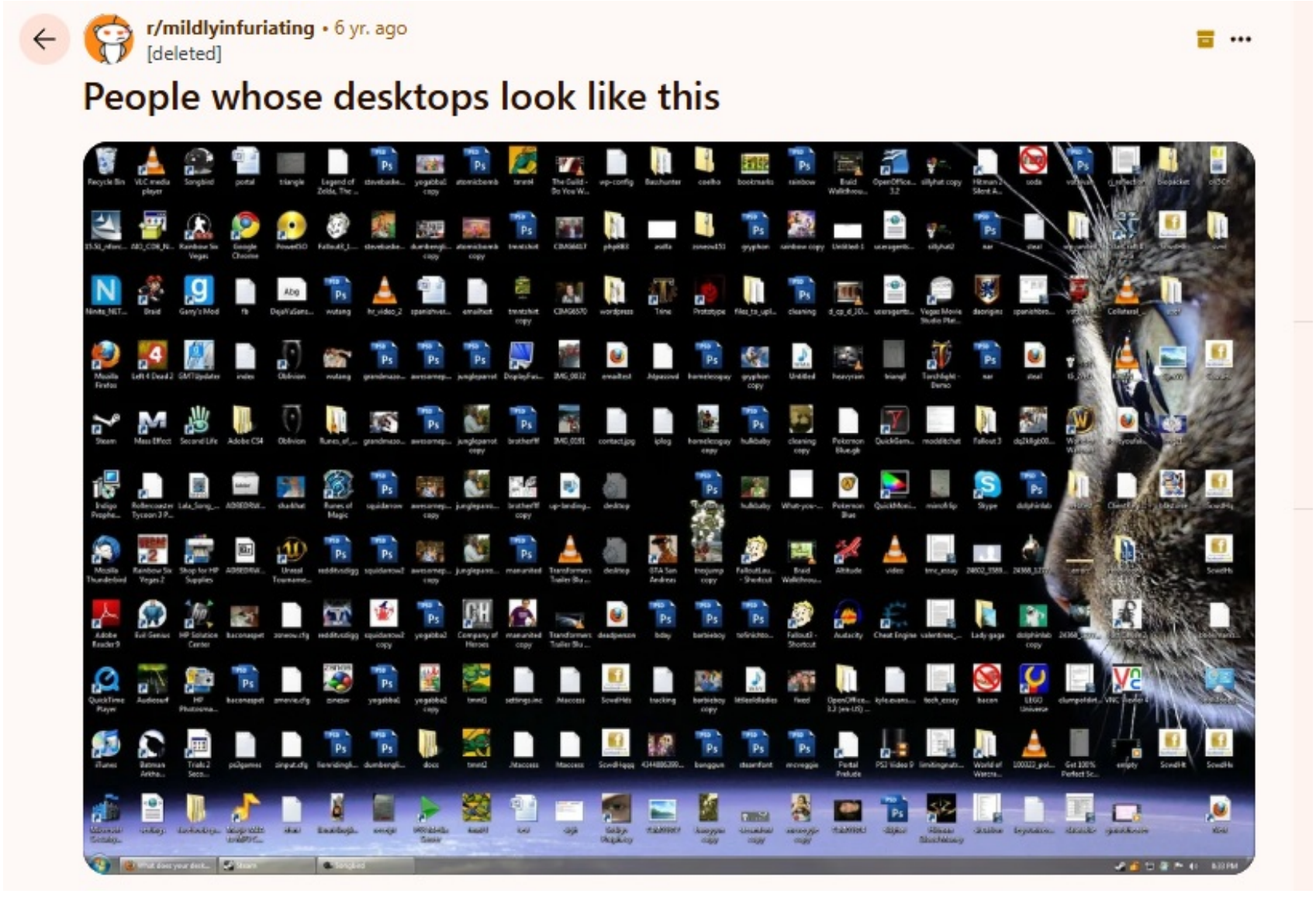


The Command Line

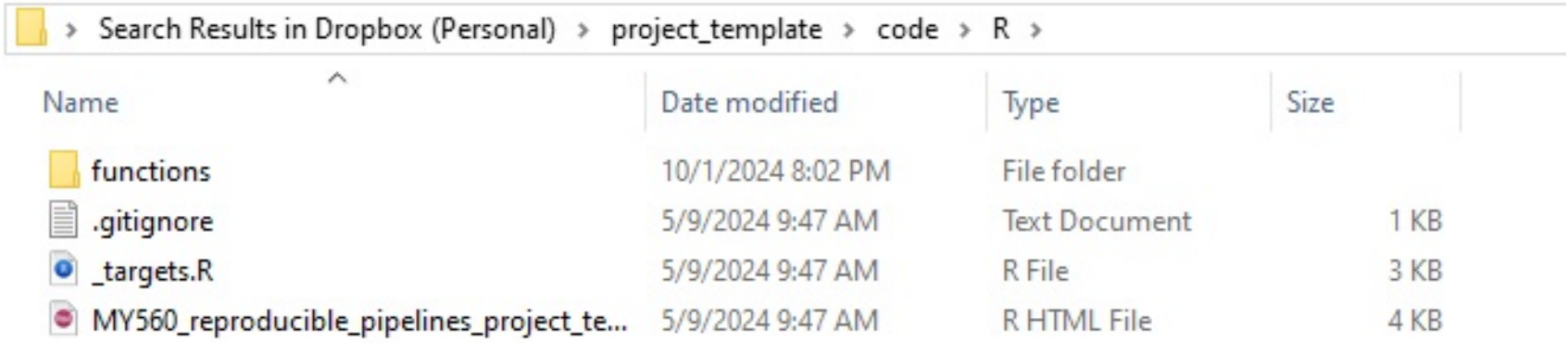
Useful commands (Linux/Unix/MacOS):

- `pwd`: Print working directory
- `ls`: list folders and files in the working directory
- `cd`: Change directory using relative filepaths
  - `cd ..` goes up one folder level
- Other helpful commands include `mkdir`, `rmdir`, `rm`, and `touch`
- Can also use `git` in the command line (more later!)

Folders and Files



Folders and Files



Folders and Files

Some general rules to follow:

- For each project you work on, create a new folder to house all files related to that project
- If using Git/GitHub (more later!), each repository = a project = a top-level folder (with optional sub-folders)
- You can have too much structure, but more > less
- Use descriptive names for folders and files, and avoid spaces in file names:
  - Folders: `scripts`, `data`, `results`, `figures`, `docs`, `code`, `notebooks`
  - Files: `data_raw.csv`, `data_clean.csv`, `runner.py`, `cleaning.R`, `results.txt`, `figures.png`, `report.qmd`
  - Avoid hardcoding dates or versions where possible (sometimes necessary, but better inside the file)

Folder and Files

**footnote: Thanks to Matt from UCLA for this comic idea!

Programming for Data Science

Programming Languages: Overview

A programming language has:

1. Primitives:
   - ➜ Data types: Numbers, strings, etc.
   - ➜ Operators: Mathematical, logical, etc.
   - ➜ Control flow: If-else statements, loops, etc.
2. Syntax:
   - ➜ Words: Individual tokens
   - ➜ Phrases: How to put tokens together
   - ➜ Context: How to assign meaning to tokens
3. Semantics:
   - ➜ What code 'means' to your computer

Programming Languages: Overview

A programming language has:

1. Primitives:
   - ➜ Data types: Numbers, strings, etc.
   - ➜ Operators: Mathematical, logical, etc.
   - ➜ Control flow: If-else statements, loops, etc.
2. Syntax:
   - ➜ Words: Individual tokens
   - ➜ Phrases: How to put tokens together
   - ➜ Context: How to assign meaning to tokens
3. Semantics:
   - ➜ What code 'means' to your computer

Computer programme: A set or series of instructions that a computer is asked to perform (aka an algorithm).

Programming Languages: Overview

A programming language has:

1. Primitives:
   - ➜ Data types: Numbers, strings, etc.
   - ➜ Operators: Mathematical, logical, etc.
   - ➜ Control flow: If-else statements, loops, etc.
2. Syntax:
   - ➜ Words: Individual tokens
   - ➜ Phrases: How to put tokens together
   - ➜ Context: How to assign meaning to tokens
3. Semantics:
   - ➜ What code 'means' to your computer

Computer programme: A set or series of instructions that a computer is asked to perform (aka an algorithm).

Remember: Your intended meaning *may not be the same* as the computer's interpreted meaning!

Programming Languages: Differences

Some points of difference:

- Low level (e.g. assembly or `C++`) vs. high level (e.g. `Python`, `R`, `Julia`)
- Functional programming vs. object oriented programming (OOP)

Functional Programming vs OOP

Two paradigms:

Functional Programming	**(OOP)**
Organizes code around *functions*	Organizes code around *objects*
Functions take inputs and return outputs	Objects belong to classes with attached methods
Emphasizes vectorization and immutability	Emphasizes encapsulation, inheritance, modularity
Functions are often pure (no side effects)	Data and behavior are bundled in objects

Programming Languages: `R` and `Python`

Perhaps the two most common programming languages for data science are `R` and `Python`.

In this course we are going to use `R` and not `Python` – but the two are more similar than dissimilar.

If you hope to work as a data scientist it is worth being somewhat familiar with both.

Programming Languages: `R` and `Python`

Perhaps the two most common programming languages for data science are `R` and `Python`.

In this course we are going to use `R` and not `Python` – but the two are more similar than dissimilar.

If you hope to work as a data scientist it is worth being somewhat familiar with both.

Let's take a brief tour of the two, and then we will focus on `R` for the remainder of the course.

R: Introduction

High level programming language targeted at statistical computing

Functional programming emphasis, but can be used in an object-oriented fashion

Uses 1-based indexing (e.g. `x[1]` is the first element of `x`)

Open source, strong community, focused on econometrics and statistics

Well suited to one-off analyses - often unstable in production

Installs from https://www.r-project.org/ (current version will be something like `R 4.X.X`)

R: Objects and Assignment

Everything in R is an object. The assignment operator (<-) binds objects on the right to names on the left.

Can also use = but conventionally use <- for assignments, and = for arguments in functions.

R: Objects and Assignment

Everything in R is an object. The assignment operator (<-) binds objects on the right to names on the left.

Can also use = but conventionally use <- for assignments, and = for arguments in functions.

```
# Assign the object c(1,2,3) to the name x
x <- c(1, 2, 3)
x
```

```
[1] 1 2 3
```

R: Objects and Assignment

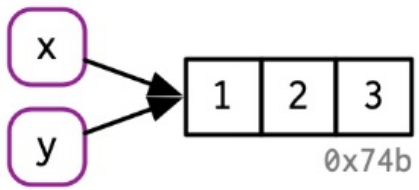
Everything in R is an object. The assignment operator (<-) binds objects on the right to names on the left.

Can also use = but conventionally use <- for assignments, and = for arguments in functions.

```
# Assign the object c(1,2,3) to the name x
x <- c(1, 2, 3)
x
```

```
[1] 1 2 3
```

```
# Assign the object c(1,2,3) to the name y, via x
y <- x
```



Source: Advanced R, Names and Values

R: Functions

Everything in R is an object, but things happen via function calls:

- Functions are first-class objects
- Functions take arguments and return values
- Many of these functions are built-in ({base})
- Sometimes you don't realise you are using one (but you are)
- You can also write your own functions (you should)

R: Functions

Everything in R is an object, but things happen via function calls:

- ➜ Functions are first-class objects
- ➜ Functions take arguments and return values
- ➜ Many of these functions are built-in ({base})
- ➜ Sometimes you don't realise you are using one (but you are)
- ➜ You can also write your own functions (you should)

In R we say that `print()` is a function, and the text in parentheses is the argument to the function:

```
print("Hello, world!")
```

```
[1] "Hello, world!"
```

R: Packages

R is often most useful when we use packages. Those include custom functions:

- Packages are denoted by {}, e.g. {base} or {dplyr} or {ggplot2}
- Functions can be masked by various packages (naming conflicts)
- To refer to specific packages with ::, e.g. `dplyr::filter()` will call `filter()` from the {dplyr} package, even if there are other functions called `filter()`

Packages are hosted on the Comprehensive R Archive Network (CRAN) and usually well documented

Install packages with `install.packages("PACKAGE_NAME")`.

Load them with `library(PACKAGE_NAME)` or `require(PACKAGE_NAME)`.

Python: Introduction

High level programming language for general computing

Object-oriented emphasis, but can be used in a functional fashion (we will mostly do this)

Uses 0-based indexing (e.g. `x[0]` is the first element of `x`)

Open source, strong community, focused on machine learning

Versatile and "robust" - good for production

Natively installed on Mac, but you might want to update. For Windows, installs from https://www.python.org/ or via command line (current version will be something like `Python 3.X.X`)

Python: Objects and Assignment

Like R, everything in python is an object.

The assignment operator (=) binds a variable (left) to an object (right) in memory.

```
# Assign the object 5 to the variable x
x = 5
print(x)
```

5

Again, the variable x points to the object 5, so could y

Python: Functions

As with R, functions play an important role in python:

- ➜ Functions are first-class objects
- ➜ Functions take arguments and return values
- ➜ Many useful functions are built-in
- ➜ You can write your own functions

```
print("Hello, world!")
```

```
Hello, world!
```

Python vs. R: Operators and Mathematical Functions

Description	Python	R
Addition	+	+
Subtraction	-	-
Division	/	/
Multiplication	*	*
Exponentiation	**	^
Modulus (remainder)	%	%%
Integer (floor) division	//	%/%

Python vs. R: Logical Operators

Purpose	Python	R
Less than	`<`	`<`
Greater than or equal to	`>=`	`>=`
Is equal to	`==`	`==`
Not equal to	`!=`	`!=`
Logical AND	`and`	`&`
Logical OR	`or`	`|`
Membership test	`in`	`%in%`

Python vs. R: OOP vs. Functional Programming

Problem: Take an input and double it.

Functional programming solution in R:

```
# Define function that takes x and doubles it
double <- function(x) {
  x * 2
}

# Use the function for an input:
double(4)
```

```
[1] 8
```

Python vs. R: OOP vs. Functional Programming

Problem: Take an input and double it.

Object-oriented programming solution in python:

```
# Create a class called Number with attached methods:
class Number:
    def __init__(self, x):
        self.x = x
    # Method to double the object
    def double(self):
        return self.x * 2

# Create an instance of the Number class
num = Number(4)
# Use the method to double the object num
num.double()
```

Python vs. R: OOP vs. Functional Programming

Problem: Take an input and double it.

But python can also be used functionally:

```
# Define function that takes x and doubles it
def double(x):
    return x * 2

# Use the function for an input:
print(double(4))
```

```
8
```

Python vs. R: Conventions

Python has strong conventions and 'best' practices:

- ➜ Consult the PEP 8 style guide
- ➜ People talk about code being `pythonic` or not

Python vs. R: Conventions

Python has strong conventions and 'best' practices:

- Consult the PEP 8 style guide
- People talk about code being `pythonic` or not

R has far fewer conventions:

- Consult R for Data Science (2e) for the closest approximation to PEP 8
- People talk about code being either `tidy` or `base` (*not* based) - this is very different to what `pythonic` means

Aside: SQL and Database Languages

We won't learn any, but you should know about Structured Query Language (SQL)

SQL comes in many flavours, but broadly it is used to query and manipulate databases

Reasonably easy to learn, but very powerful

We will learn `{dplyr}` in `R` which often mocks SQL by design.

```sql
SELECT first_name,
       last_name,
       email
FROM employees
WHERE department = 'Sales'
```

Markup Languages: Markdown and Dynamic Documents

Markdown: Plain text with formatting.

Can be used in notebook fashion, combining chunks of `R` ('Rmarkdown') or `Python` ('Jupyter') code with text – these are called dynamic documents.

We will use `.qmd` files that combine plain text and `R` code chunks. You can also use other languages in the same file.

Your seminar/class exercises will be run from .qmd files, focused on `R`.

Markup Languages: Markdown and Dynamic Documents

````
## Part 2: Data Types in R
#### 6. Exercise

Lists are used to store multiple items, not necessarily of the same type, in a single variable. In R, we create them with `list()`.

* Ordered
* can contain arbitrary objects
* Elements can be accessed by index (`[[ ]]`) or by name (`$`)
* Allow duplicates
* Can be nested
* Can be modified after creation

▷Run Cell | Run Next Cell | Run Above
```{r}
my_list <- list("dog", "cat", 1, 1.521, TRUE)
my_list
```



## Part 3: Functional Programming and Control Flow

__Functional Programming:__

As we learned in the lecture, everything in R is an object, but most things happen via function calls. A lot of packages in R come with their own customized functions, but you can also define your own function.

Let us define a small helper function for us to work with:
We want to write a R function `triple()` that takes a single numeric `x` and returns `x * 3`.

▷Run Cell | Run Next Cell | Run Above
```{r}
triple <- function(x) {
  x * 3
}

triple(4)
```
````

Integrated Development Environments (IDEs)

One can just write code in a text editor (e.g. notepad), but why would you. . .

Integrated Development Environments (IDEs)

One can just write code in a text editor (e.g. notepad), but why would you...

Instead, use an IDE which provides comfort and productivity tools.

Examples (there are many):

- RStudio: For `R` and `Rmarkdown`
- PyCharm: For `python`
- VScode: For many languages and dynamic documents
- **Positron**: A fork of VScode that integrates many of the strengths of RStudio


Using Positron

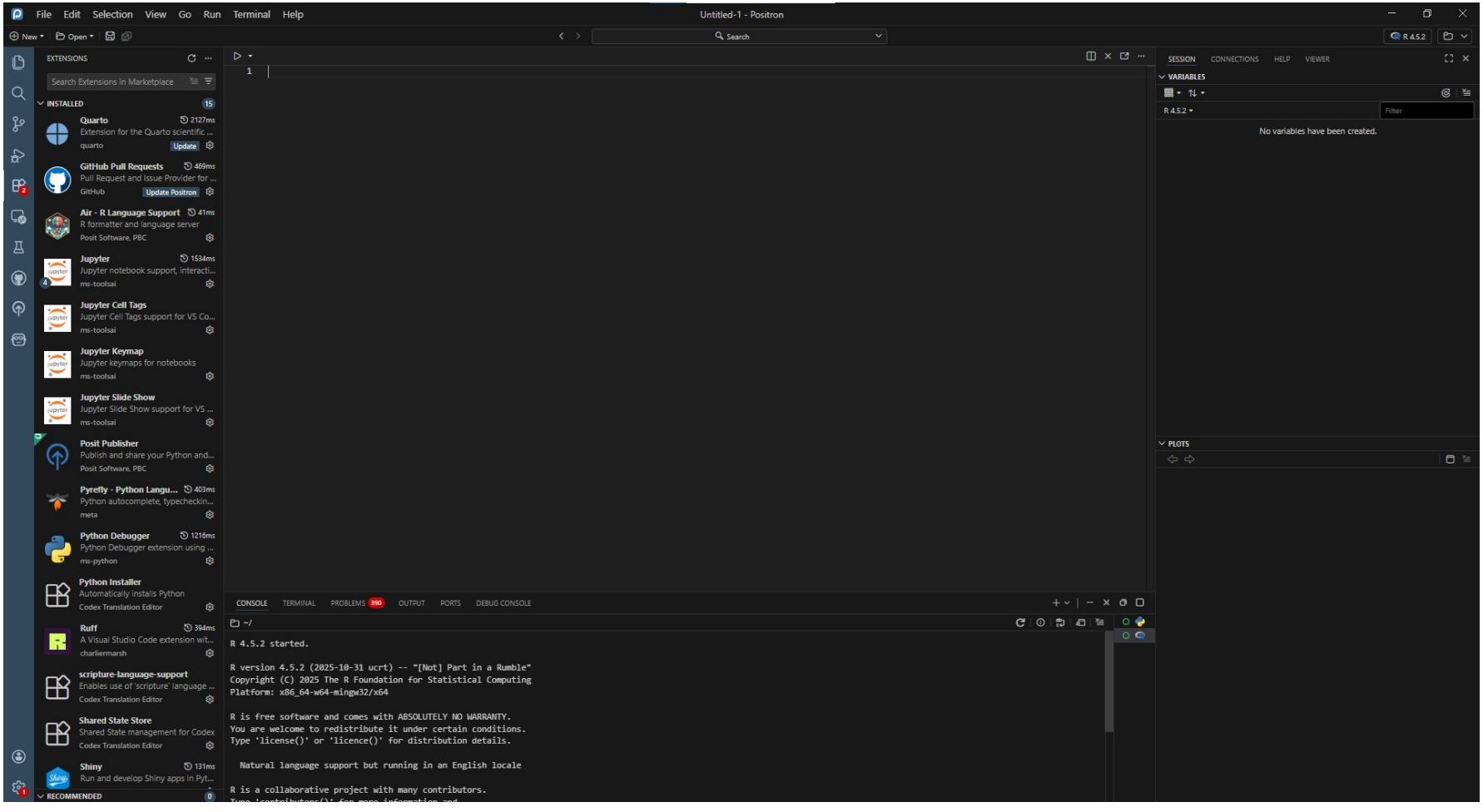
Install from: https://positron.posit.co/download.html/

Positron supports many 'extensions' which you can install from within the IDE:

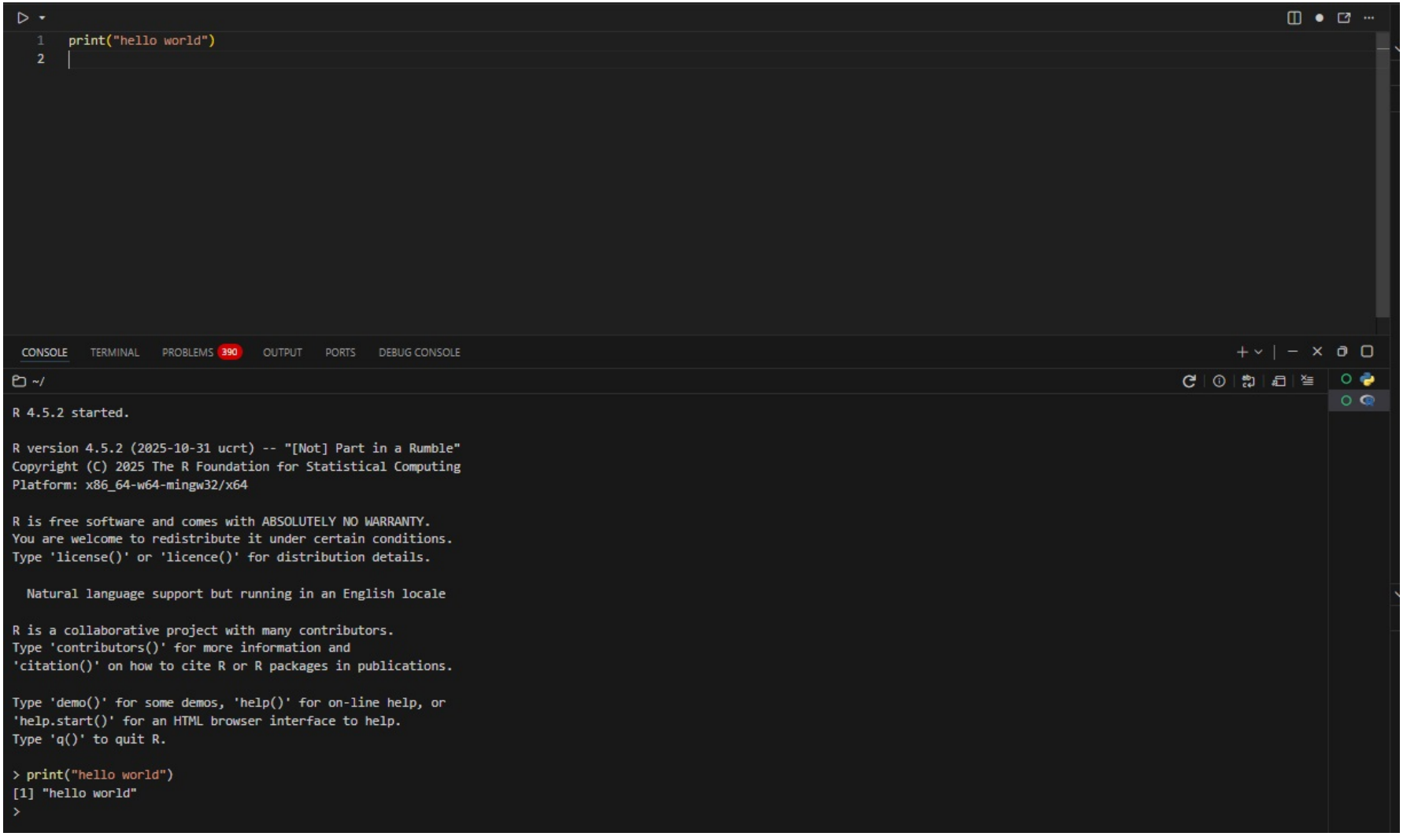
- `R` extensions
- `python` extensions
- `Quarto` extension
- `GitHub Pull Requests` extension

You may need be prompted to install some packages or modules on startup (e.g. `{languageserver}` for `R`)

Using Positron

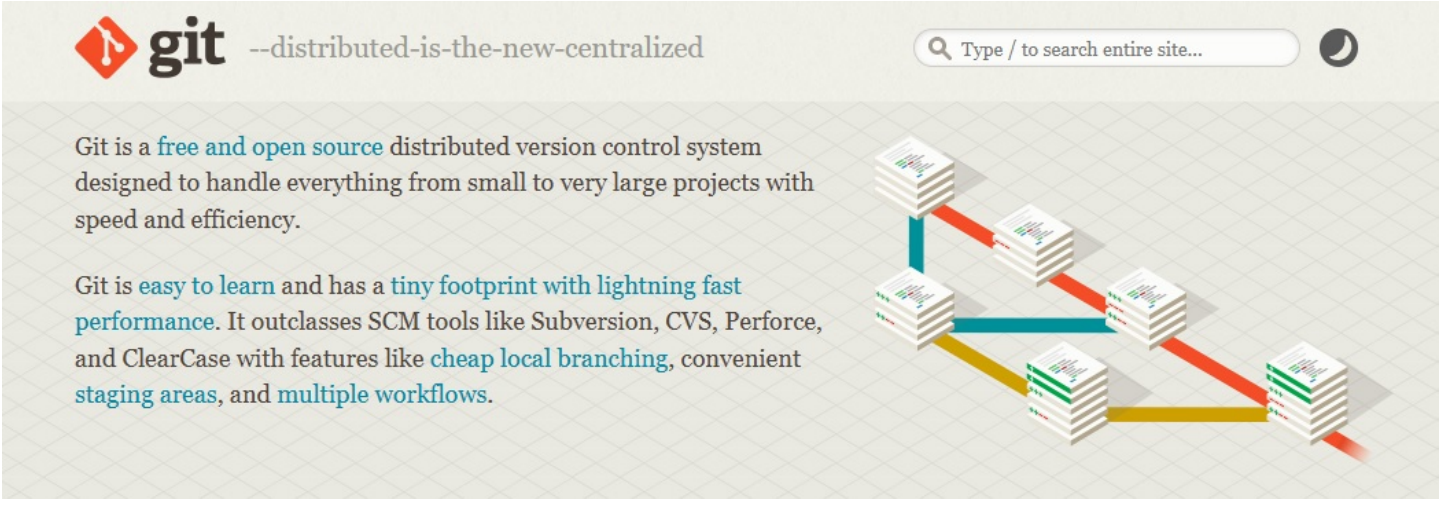


Using Positron

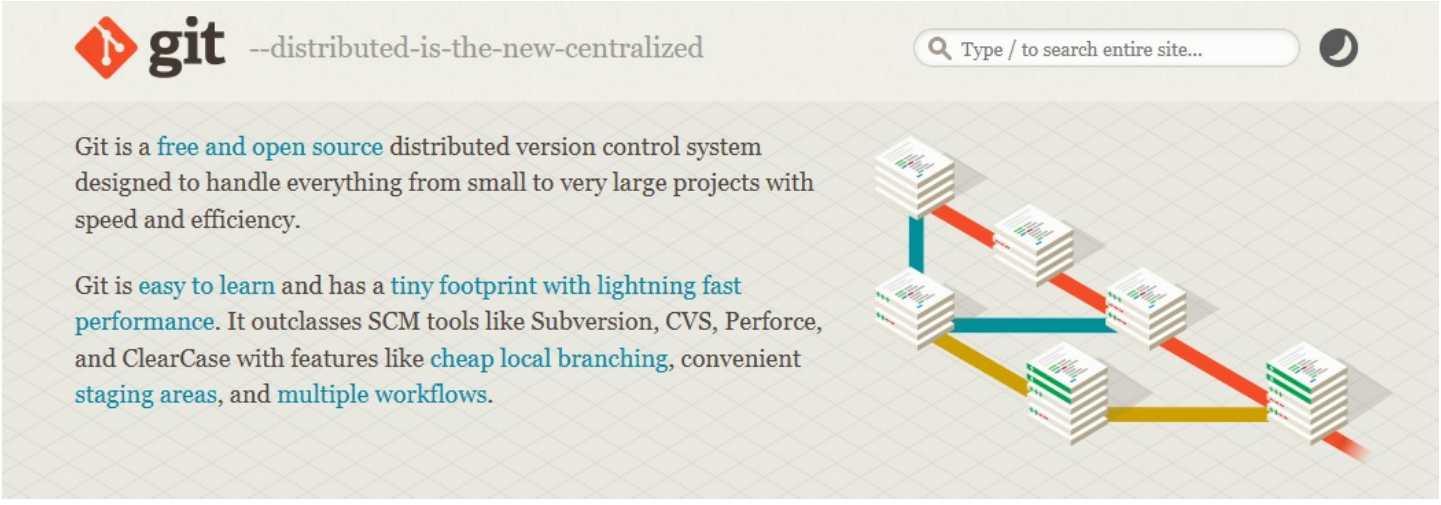


Version Control

Git


git
--distributed-is-the-new-centralized
Type / to search entire site...
Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.
Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

Git

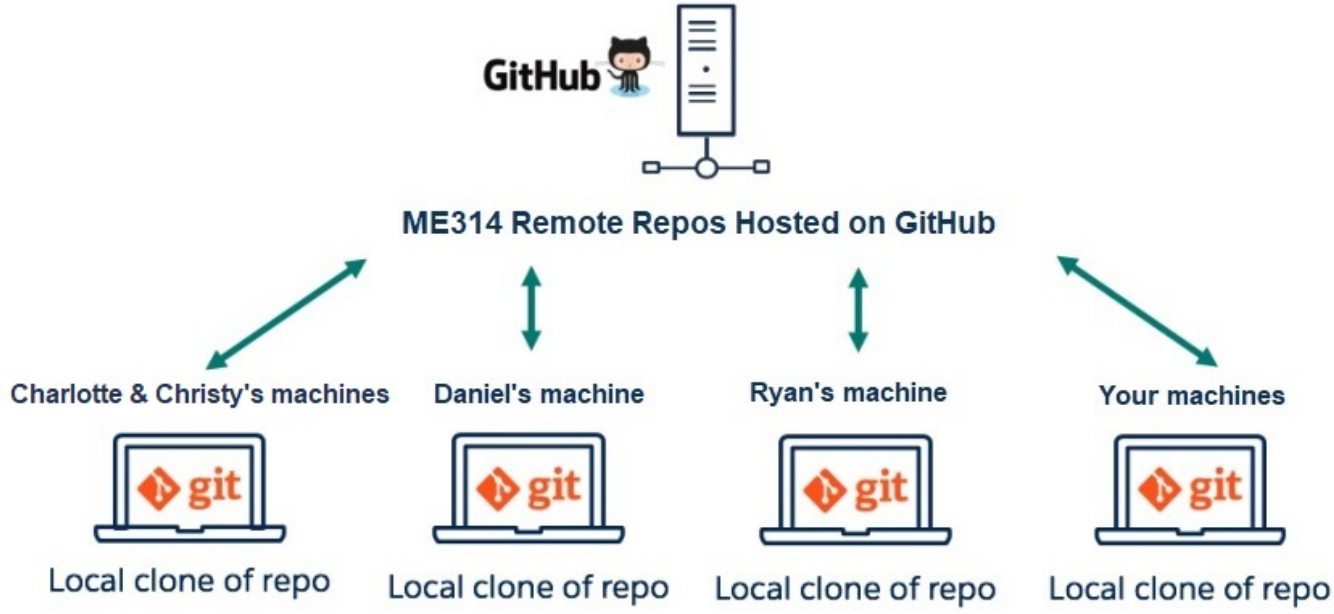


Version control system designed to (collaboratively) manage files.

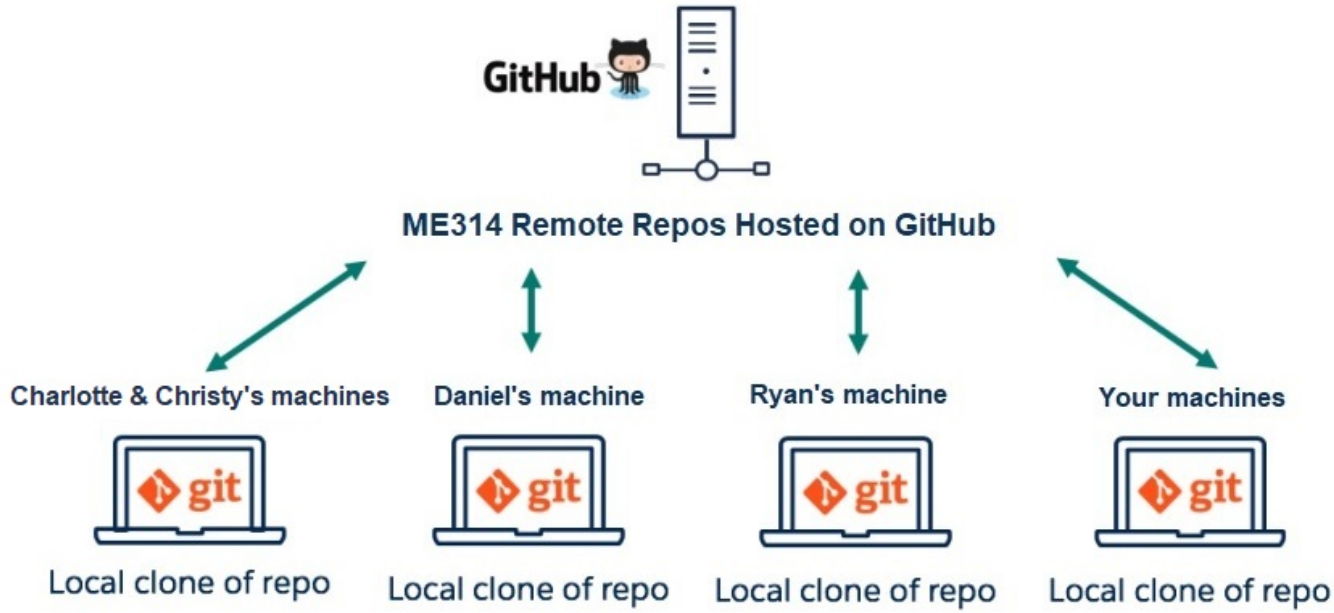
Git "tracks" files and any changes made to it over time – you can always seen *who* made *what* changes *when*.

If something bad happens, can revert to previous versions.

Git: Repositories



Git: Repositories



Files are typically grouped into repositories – a folder in which all files are tracked (unless ignored)

This can either occur locally or in distributed fashion (over multiple machines/servers)

Git is an indisputably necessary part of any professional data science environment

GitHub in ME314

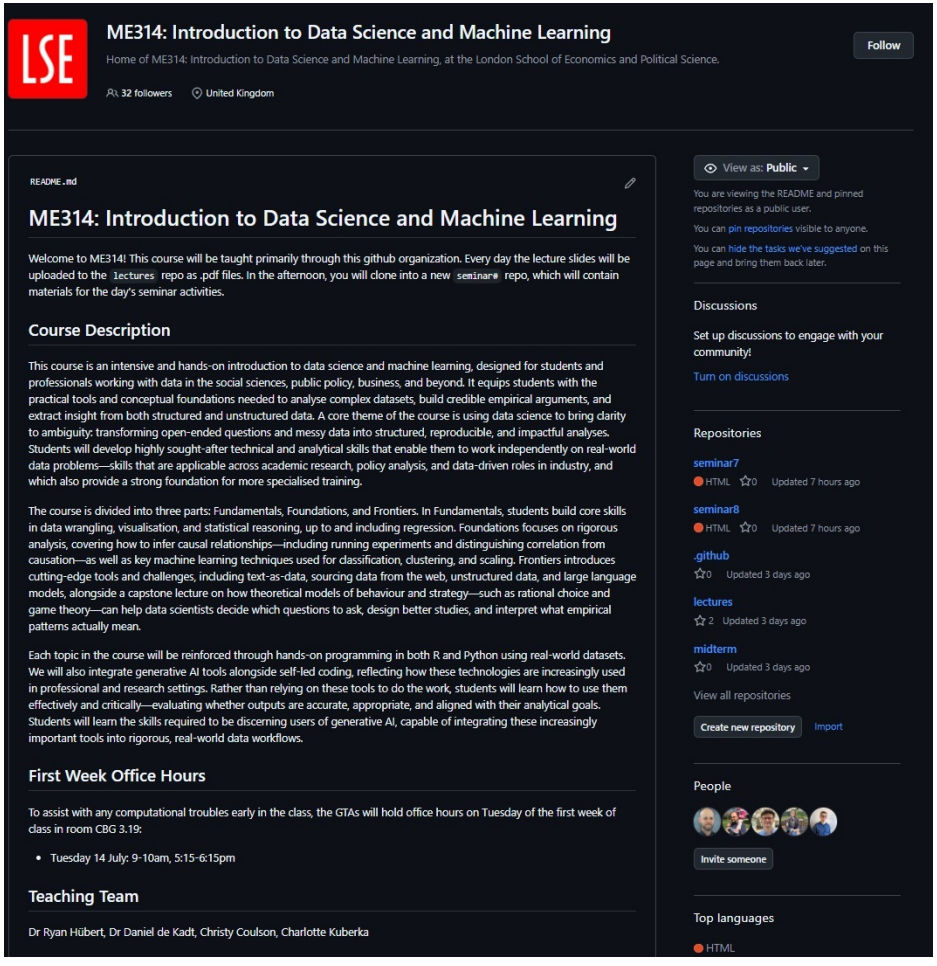
GitHub is a service for managing Git, collaboration, and version control

We will use GitHub (and thus Git) extensively in this class

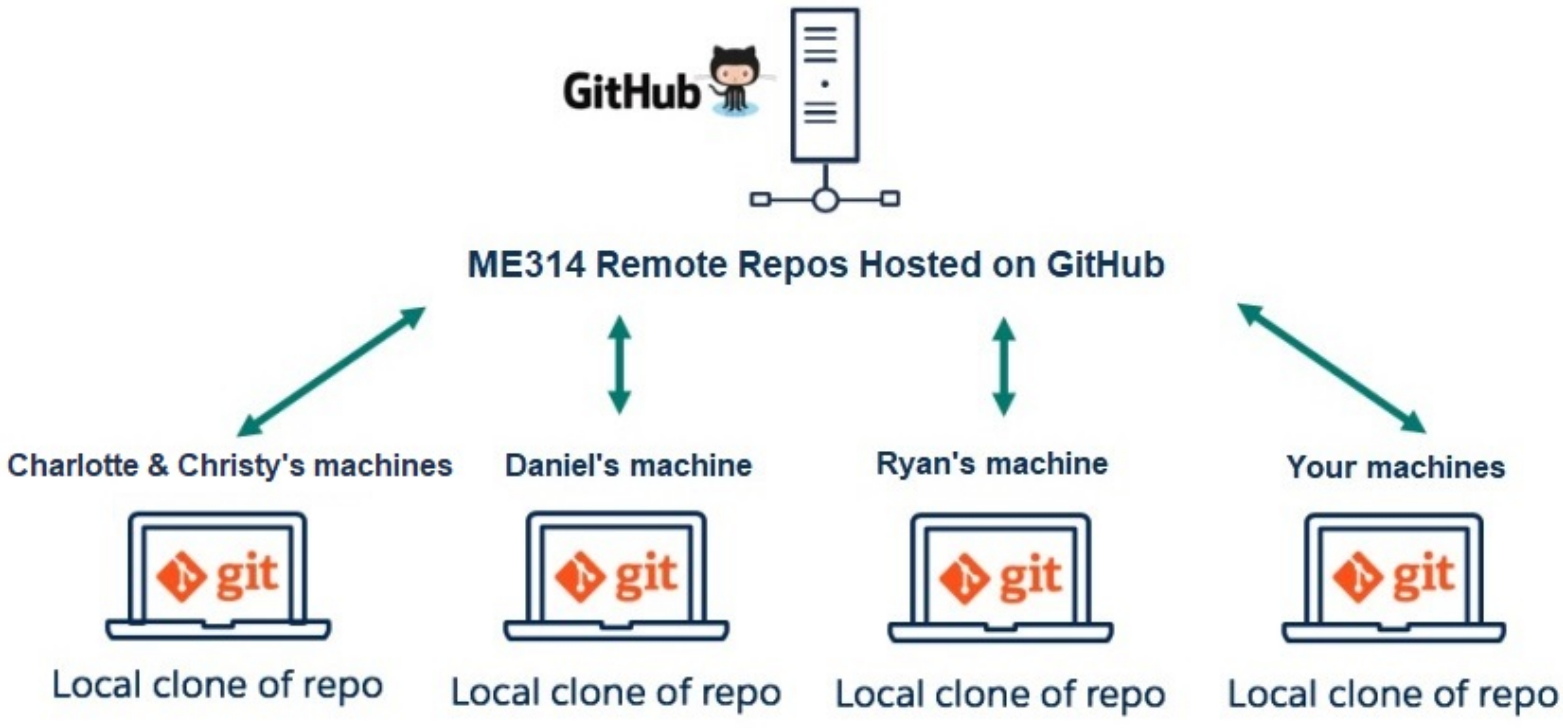
The course is hosted on GitHub:

1. Organization page: https://github.com/me314-lse.
2. Lecture slide repo: https://github.com/me314-lse/lectures.
3. Seminars each have their own repo:
    a. Seminar 1
    b. Seminar 2
    c. Etc.

GitHub in ME314

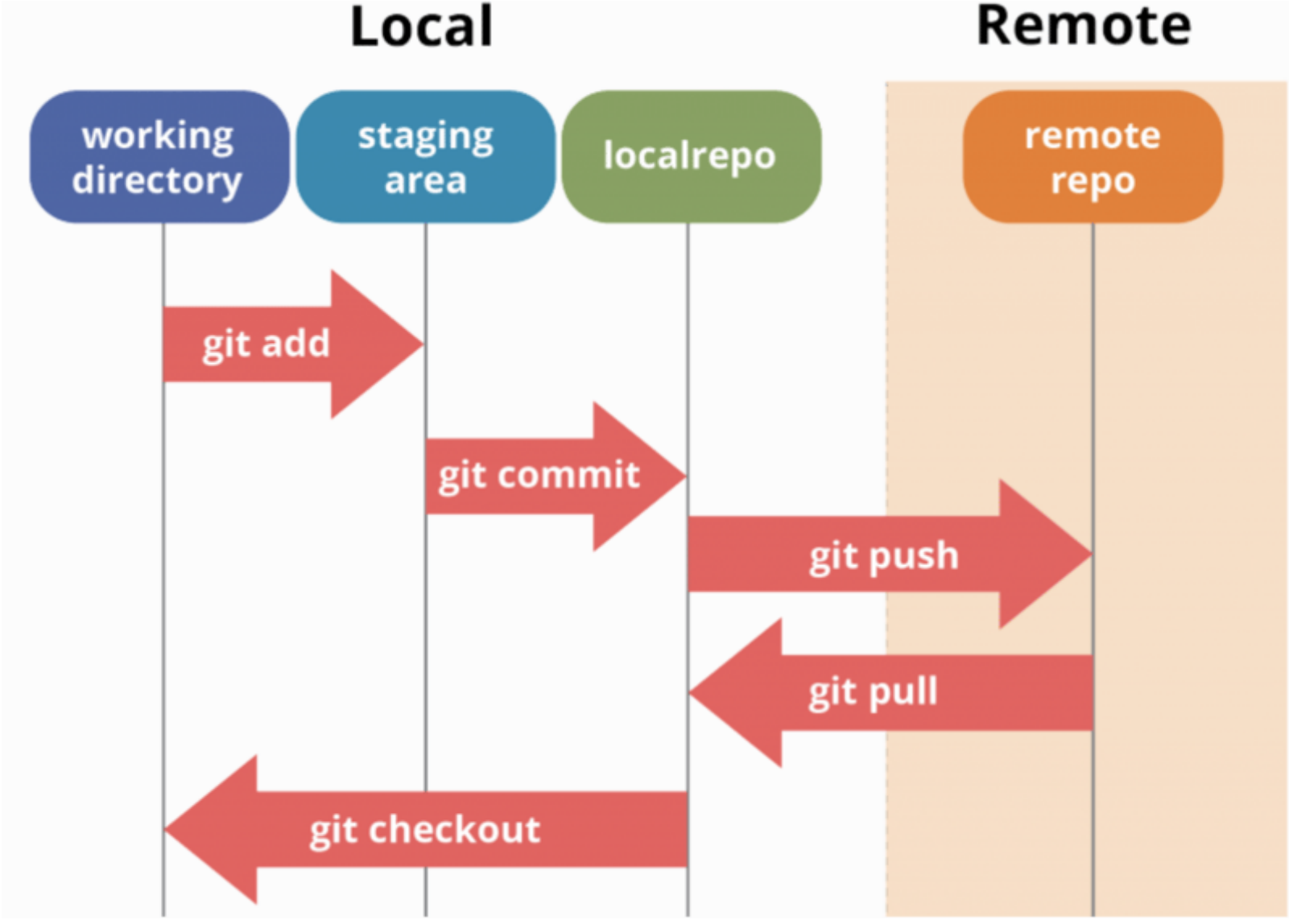


GitHub in ME314



Source: Adapted from sfdc technie, GitHub

GitHub: Workflow Basics



Source: Molly Nemerever, Git, GitHub, & Workflow Fundamentals

GitHub: Workflow Commands

For our purposes, the key commands we will use throughout are:

1. `git clone`: Save a repo from a remote server to a local folder
2. `git pull`: Pull files from the remote server to the local folder
3. `git status`: Check the status of the local repo
4. `git add`: Stage files, preparing them for commit
5. `git commit`: Commit changes to the local repo, with a message
6. `git push`: Push changes and message from the local repo to the remote server

To use, navigate to a local repo on your computer, and use these commands either in the command line (e.g. through Git Bash), or through an IDE (VScode, or GitHub Desktop).

We recommend using the command line while you are learning ('learn to fly!').

GitHub: Advanced Usage

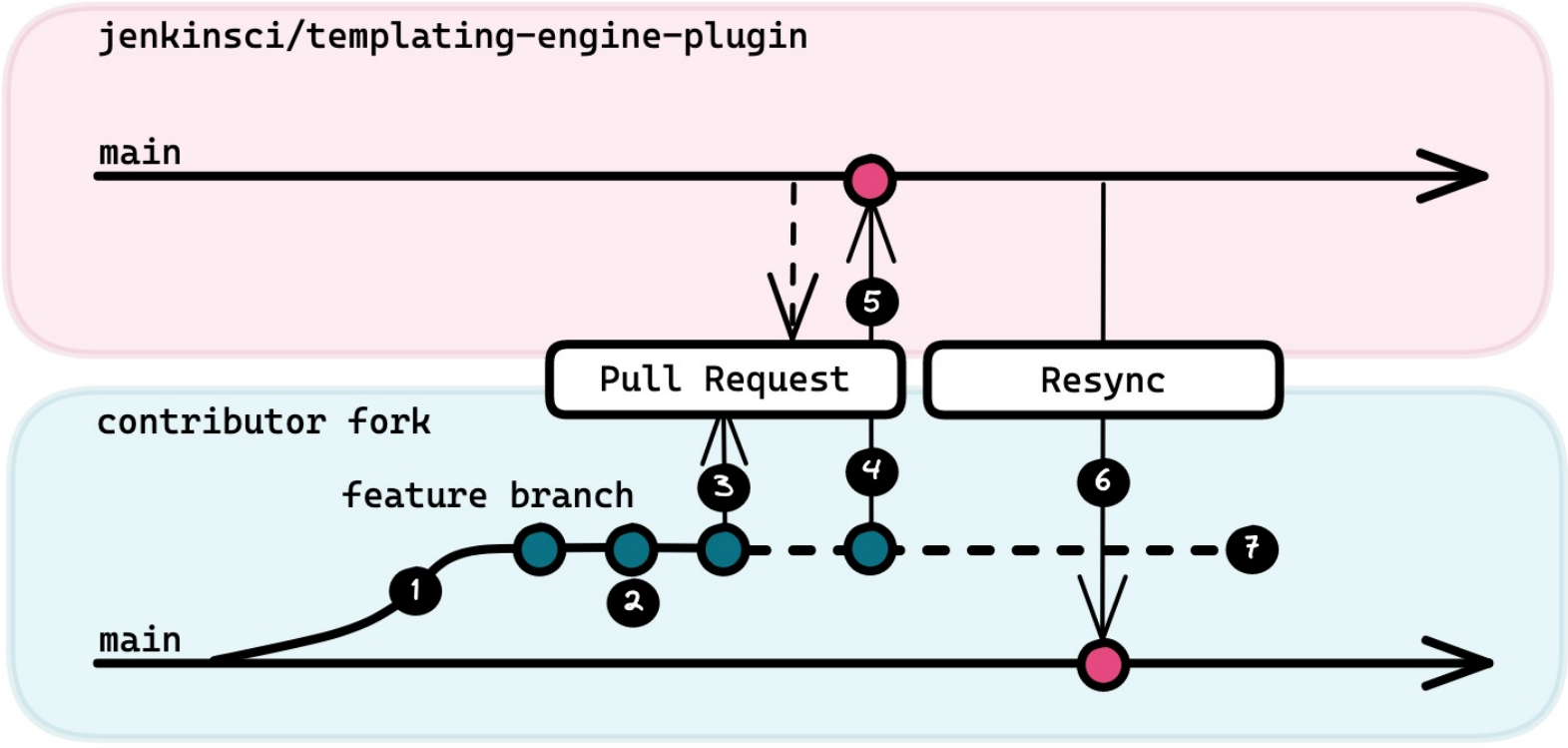
In a development setting, you will also need to understand and use:

- Fork: Make your own version of a different repository (you're *forking* off, not *cloning*)
- Branch: A parallel version of the code originating at a particular point
- Merge: Combining branches together to maintain single codebase
- Pull request: GitHub based request to merge a branch or a fork into other code

You won't use these in this course, but they are important for professional data science work.

A great site for practice: https://learngitbranching.js.org/

GitHub: Advanced Usage



Source: Jenkins Template Engine, Fork-Based Contribution Model

Summary and Next Steps

Today we covered:

1. What data science is, what kinds of problems data scientists solve with what solutions
2. What programming languages are, and the very basics of `R` and `python`
3. What version control is, and how to use Git and GitHub

This afternoon you will practice 2 and 3.

Tomorrow:

1. What is data?
2. Different kinds of data
3. Working with data in `R`